

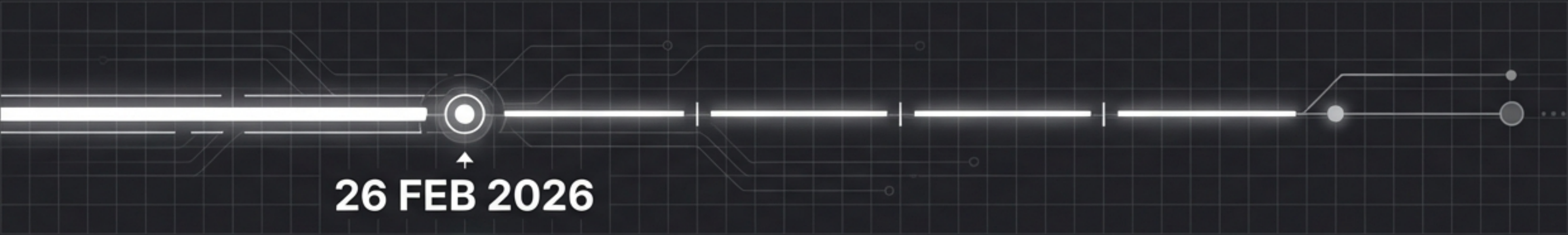


Proven AI Sender Pipeline: End-to-End Encrypted Document Delivery

Arthitecture Validation v0.6.32 | 26 February 2026

The Breakthrough: Zero-Knowledge Delivery Achieved

● SYSTEM STATUS: VERIFIED



A **Claude.ai** session successfully created, **encrypted**, and emailed a **PDF**.
The server never saw the **plaintext**, **filename**, or **key**.



Source
Sherpa Agent via
SGraph Send



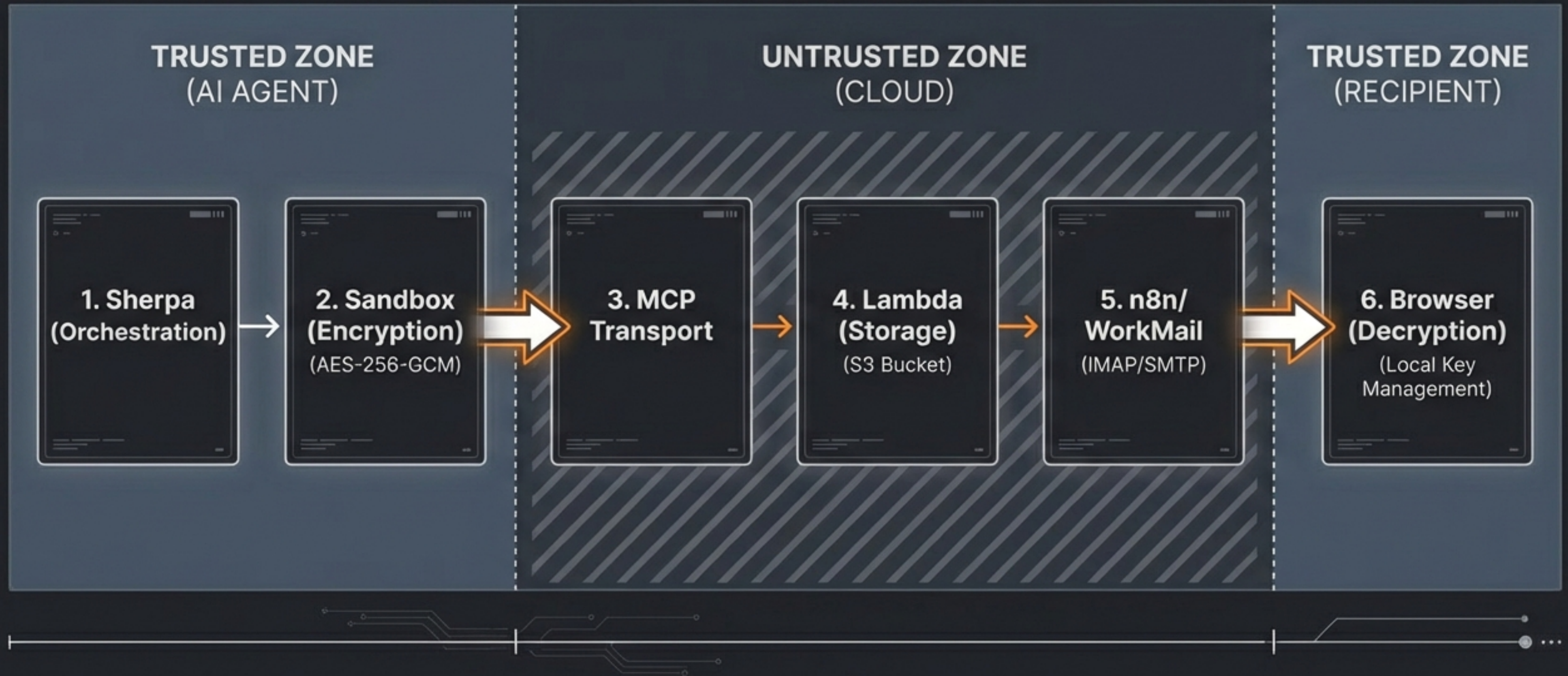
Mechanism
AES-256-GCM
(Sandbox)



Outcome
Zero-Knowledge
Property Maintained

The Pipeline Architecture

● SYSTEM STATUS: SECURE PIPELINE ACTIVE



The Orchestrator & The Engine



Sherpa Agent

System prompt with MCP workflow instructions.

Performs step-by-step error handling and mandatory verification.



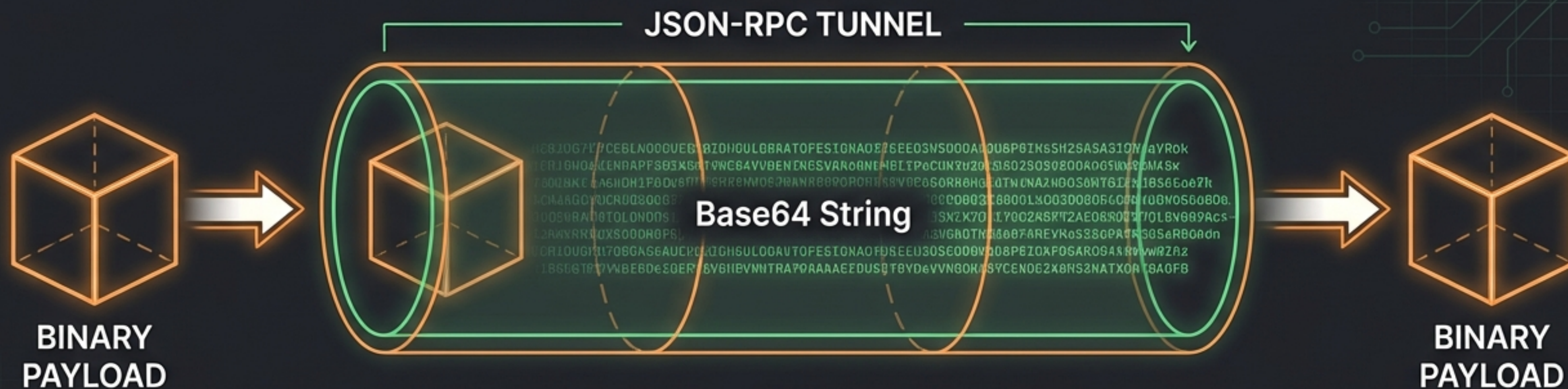
Claude Sandbox

Python runtime inside the session.

Uses ``cryptography`` for AES-256-GCM and ``reportlab`` for PDF generation.

Security Critical: Key generated via ``os.urandom(32)``. The key never leaves the sandbox except as a URL fragment.

Solving the Transport Challenge: SGraph Send MCP

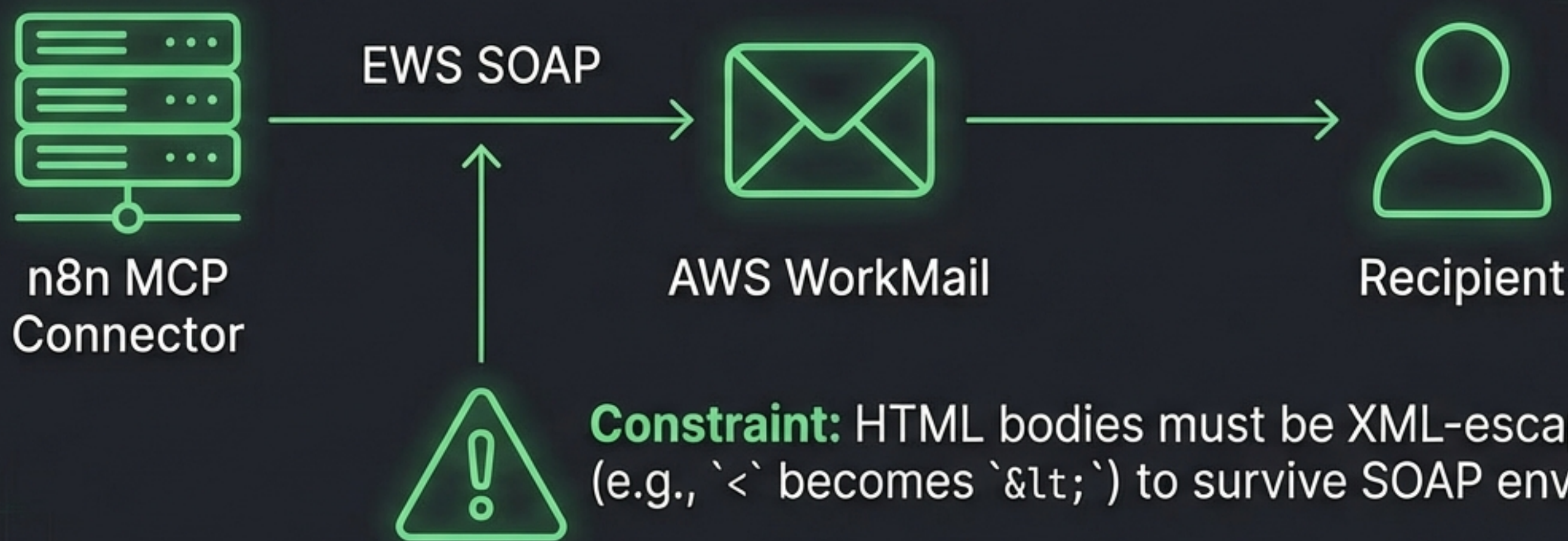


The Problem: MCP tools use JSON-RPC. Binary payloads cannot travel in the HTTP body.

The Solution (v0.6.32): The `transfers\_upload` tool utilizes a `data` parameter.

Mechanism: Payload is Base64-encoded ciphertext passed as a string argument. Server decodes back to raw bytes for storage.

The Delivery Mechanism: n8n & WorkMail



Security Guarantee: The email body contains ONLY the download URL. The decryption key exists solely in the URL fragment.

The Zero-Knowledge Guarantee

WHAT THE SERVER SEES

- ❖ Ciphertext (Opaque Binary Blob)
- ❖ Timestamps (Created, Uploaded)
- ❖ Hashed IP (SHA-256 + Salt)
- ❖ Transfer ID (Random 12-char string)

WHAT THE SERVER **CANNOT** SEE

- ❖ File Content (Encrypted at edge)
- ❖ Filename (Hidden in SGMETA envelope)
- ❖ Decryption Key (Lives in URL fragment)



Security Proof: Ingress & Envelope

Proof Point 1: Encryption Before Upload.

The Sandbox executes encryption BEFORE calling `transfers\_upload`.

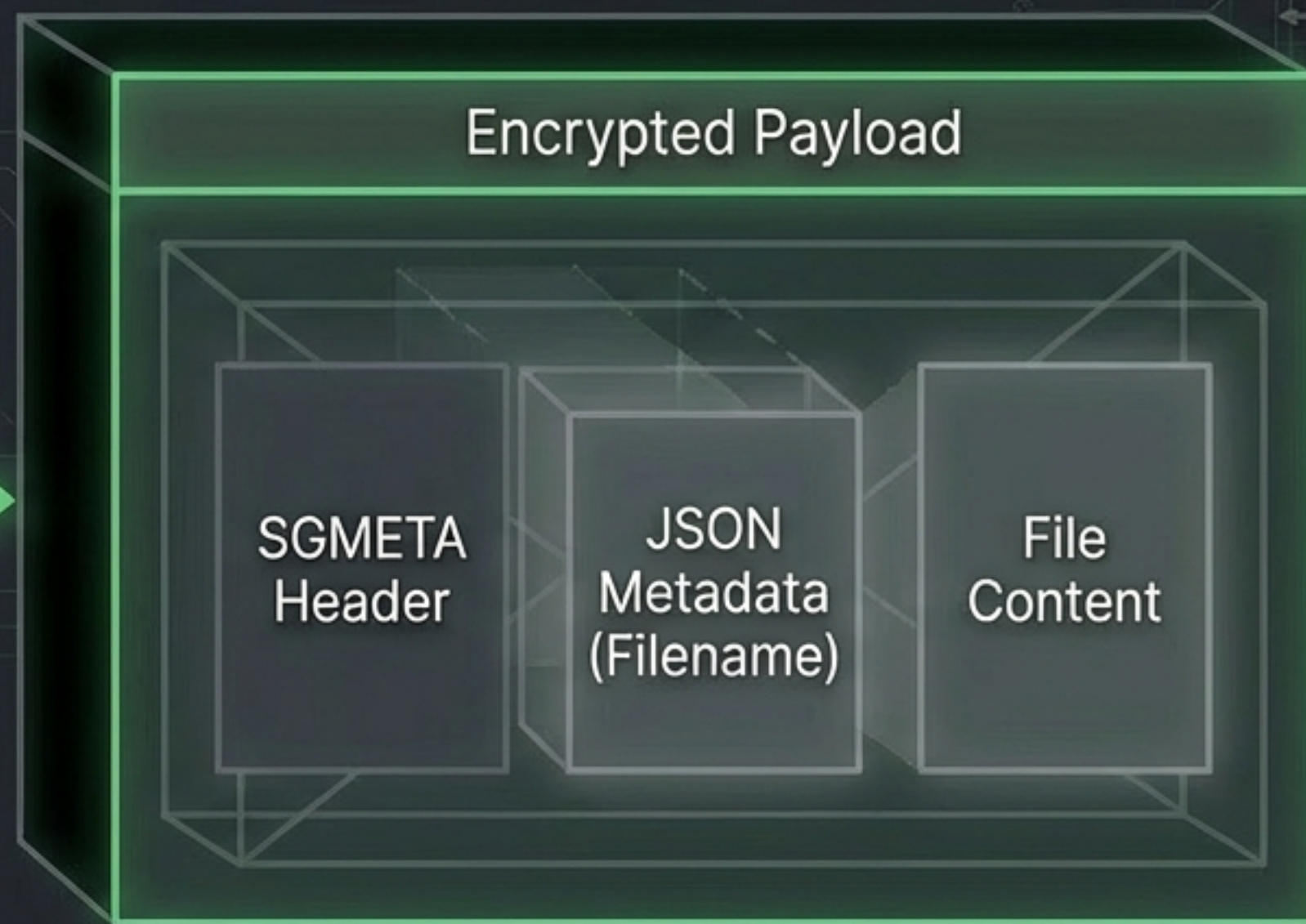
AES-256-GCM



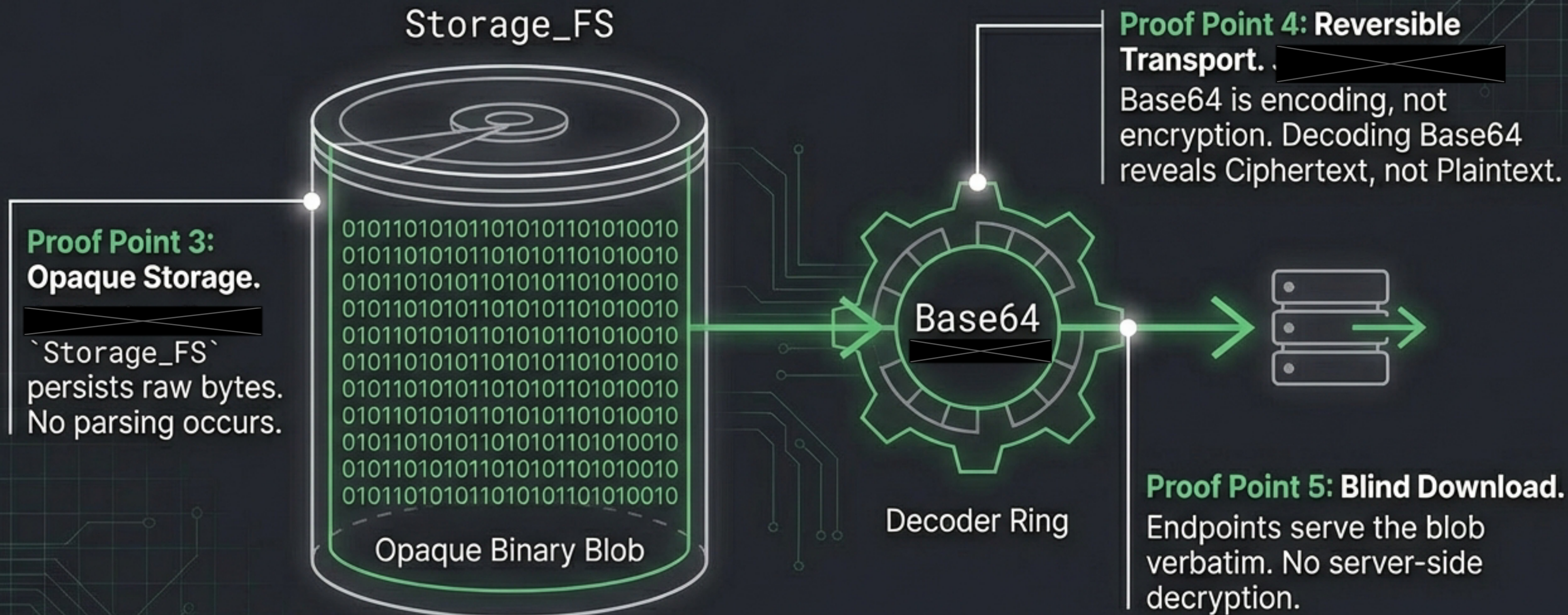
Proof Point 2: SGMETA Envelope.

The filename is inside the envelope. The entire envelope is encrypted, so the server cannot extract metadata.

Exploded View



Security Proof: Egress & Storage



The Anatomy of the Download Link

`https://host/path/
download.html#key=dGhpc0lz...`

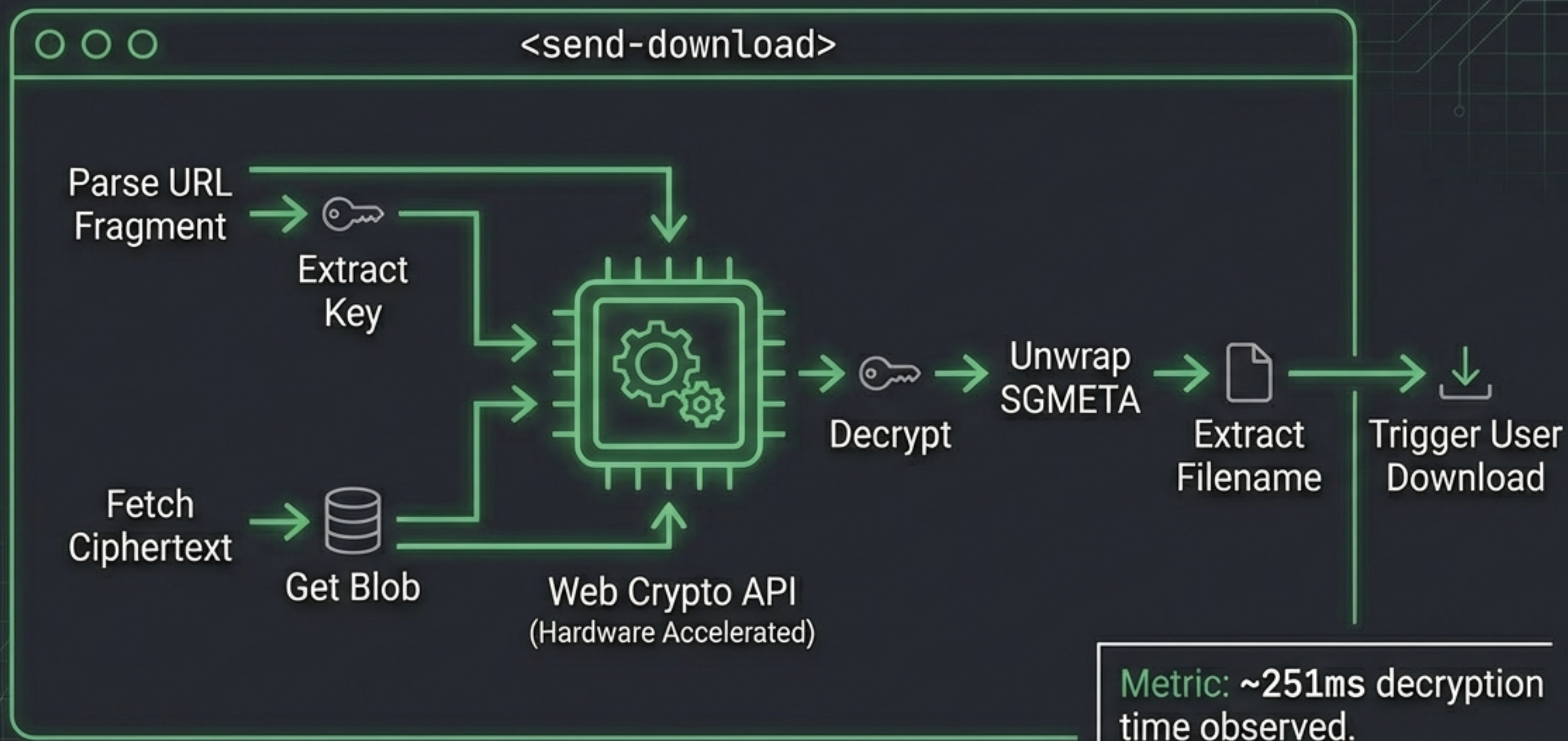
Visible to Server
(HTTP Request)

THE WALL
(Fragment Separator)

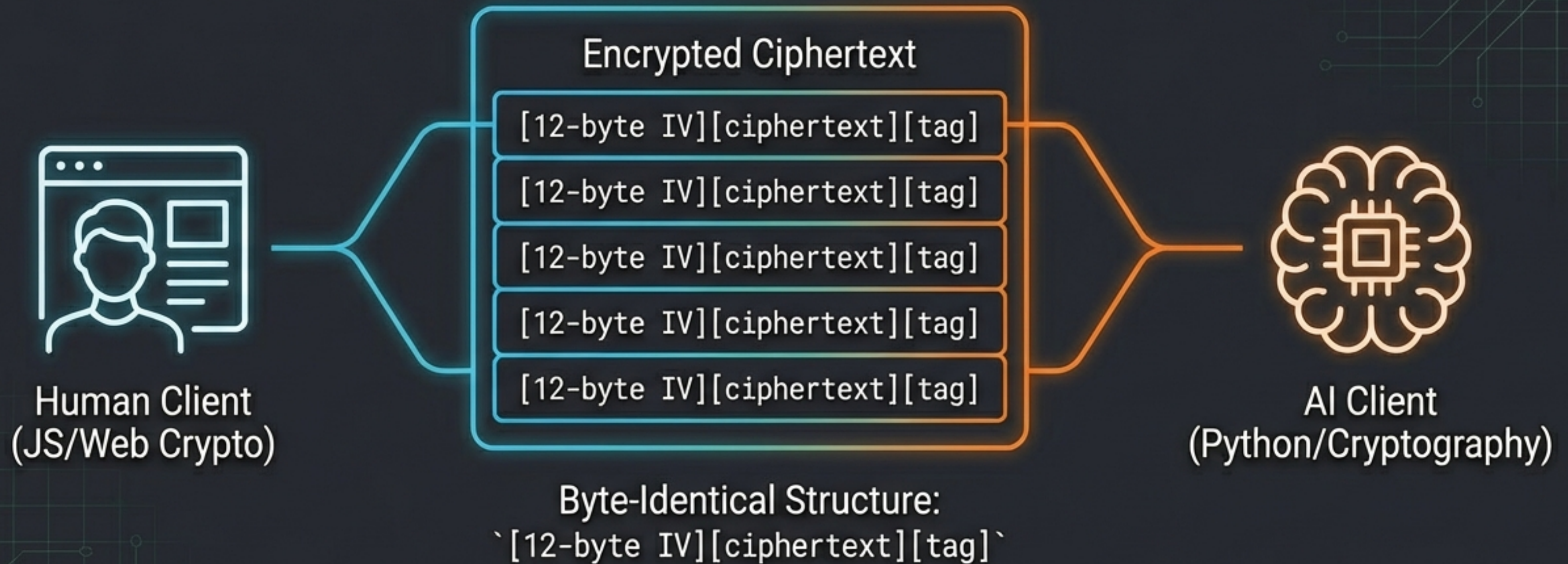
Invisible to Server
(Client-Side Only)

Validation: RFC 3986 Section 3.5 confirms fragments are never sent in HTTP requests.

Client-Side Decryption



Confirmed Parity: AI vs. Human



The server cannot distinguish between a file sent by a human and a file sent by an AI. AI Agents are first-class citizens.

Architectural Decisions Proven



SGMETA Envelope. Proven. AI-uploaded filenames survived the round trip.



IV Compatibility. Proven. Python cryptography and JS Web Crypto interact perfectly.



MCP Projection. Proven. REST API routes mapped 1:1 to MCP tools.



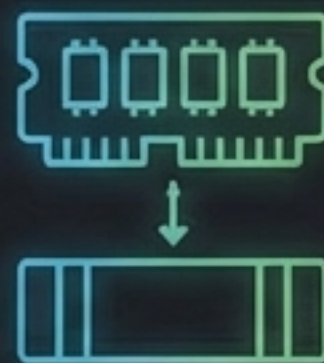
Lambda Compatibility. Proven. Streamable HTTP transport works without SSE.

Known Limitations (v0.6.32)



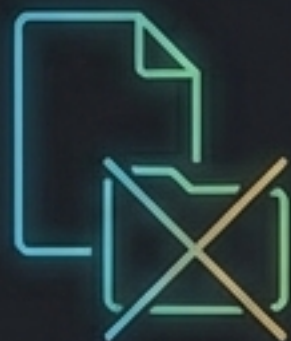
File Size

MCP Base64 path limited to ~3.75 MB (JSON constraints). Larger files require Presigned URLs.



Memory

No streaming. AI client must buffer full file before encryption.



Scope

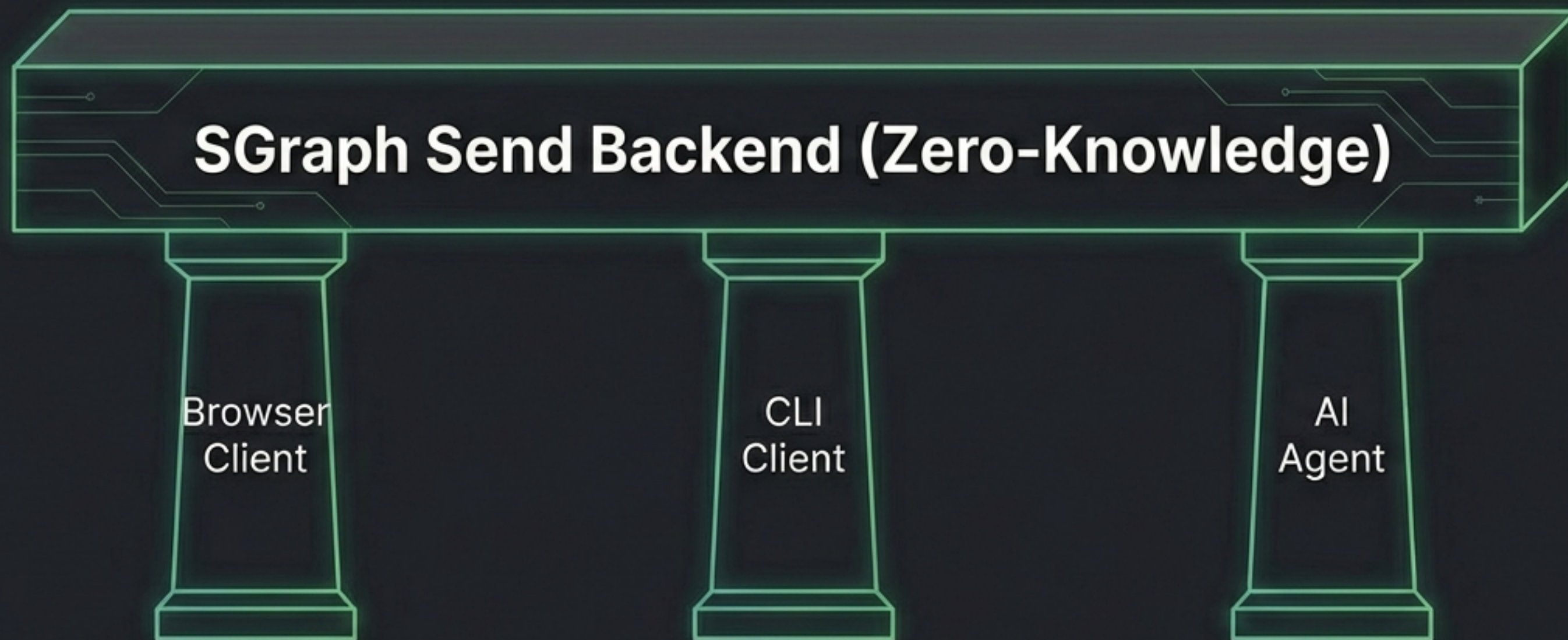
Single file only. No native folder/archive support.



Crypto

Locked to AES-256-GCM. No algorithm negotiation.

Future State: Three Clients, One Backend



The Zero-Knowledge property holds across all interfaces.
The **architecture** is validated for the Explorer phase.